Programming Assignment 2
Distance Vector Routing Algorithm

1. **<u>ALGORITHM OVERVIEW</u>**
    a. **Network Topology**
        i. Figure 1:
        ii. Node 0 connects to: Node 1 (cost 1), Node 2 (cost 3), Node 3 (cost 7)
            Node 1 connects to: Node 0 (cost 1), Node 2 (cost 1)
            Node 2 connects to: Node 0 (cost 3), Node 1 (cost 1), Node 3 (cost 2)
            Node 3 connects to: Node 0 (cost 7), Node 2 (cost 2)
    b. **Expected Convergence Results**
        i. From 0: to 0 (0), to 1 (1), to 2 (2), to 3 (4)
        ii. From 1: to 0 (1), to 1 (0), to 2 (1), to 3 (3)
        iii. From 2: to 0 (2), to 1 (1), to 2 (0), to 3 (2)
        iv. From 3: to 0 (4), to 1 (3), to 2 (2), to 3 (0)
2. **<u>DESIGN & DATA STRUCTURE</u>**
    a. **Distance Table**
        i. Dt[i][j] -> cost to reach node i via neighbor j:
            1. int costs[4][4]
    b. **Link-cost Array**
        i. Direct cost to neighbors
            1. int linkCost[4] = {0, 1, 3 ,7}
    c. **Key Design Decisions**
        i. Having 9999 to represent infinity
        ii. Only directly connected neighbors exchange packets
        iii. Distance table updated only when minimum cost changes
        iv. Send update to all neighbors when own minimum cost changes
3. **<u>PSEUDOCODE</u>**
    a. **rtinit(node) - Initialisation**
        i. Initialize distance table to 9999 (infinity)
        ii. Set dt[node][node] = 0
        iii. For each directly connected neighbor n:
        iv. Set dt[n][n] = direct link cost to n
        v. Send routing packet to each neighbor containing
        vi. current minimum costs to all nodes
        vii. Print distance table
    b. **rtupdate(node, packet) - Bellman-Ford Relaxation**
        i. Receive packet from neighbor src with their mincost[]
        ii. changed = false
        iii. For each destination d (0 to 3):
        iv. newCost = linkCost[src] + packet.mincost[d]
        v. if newCost < dt[d][src]:
        vi. dt[d][src] = newCost
        vii. changed = true

viii. if changed:
ix. Compute new minimum costs across all neighbors
x. Send updated mincost[] to all directly connected neighbors
xi. Print whether table was updated
xii. Print current distance table

c. **mincosts(out[], prev[]) - Helper Function**
   i. For each destination d (0 to 3):
   ii. out[d] = min over all neighbors j of dt[d][j]
   iii. Return out[]

4. **<u>MODULE HIERARCHY CHART</u>**

a.
```
main() [prog3.c]
|
├── rtinit0()          rtinit1()          rtinit2()          rtinit3()
|   └── tolayer2()     └── tolayer2()     └── tolayer2()     └── tolayer2()
|
├── rtupdate0()        rtupdate1()        rtupdate2()        rtupdate3()
|   ├── tolayer2()     ├── tolayer2()     ├── tolayer2()     ├── tolayer2()
|   └── printdt0()     └── printdt1()     └── printdt2()     └── printdt3()
|
└── tolayer2() [prog3.c — delivers packets to destination node]
```

5. **<u>SOURCE FILE DESCRIPTION</u>**

a. **Node0.c**
   i. Contains rtinit0() and rtupdate0()
   ii. Direct link costs: to node 1 = 1, node 2 = 3, node 3 = 7
   iii. Neighbors nodes: 1, 2, 3

b. **Node1.c**
   i. Contains rtinit1() and rtupdate1()
   ii. Direct link costs: to node 0 = 1, node 2 = 1
   iii. Neighbors nodes: 0, 2

c. **Node2.c**
   i. Contains rtinit2() and rtupdate2()
   ii. Direct link costs: to node 0 = 3, node 1 = 1, node 3 = 2
   iii. Neighbors nodes: 0, 1, 3

d. **Node3.c**
   i. Contains rtinit3() and rtupdate3()
   ii. Direct link costs: to node 0 = 7, node 2 = 2
   iii. Neighbors nodes: 0, 2

6. GROUP REPORTS

a. Task Completion

All required tasks for Program 2 have been completed. The program implements a distributed, asynchronous Distance Vector routing algorithm across all four network nodes and converges to the correct shortest-path cost tables.

i. Completed Tasks:

1. rtinit0(), rtinit1(), rtinit2(), rtinit3() -> all four initialisation routines implemented and tested
2. rtupdate0(), rtupdate1(), rtupdate2(), rtupdate3() -> all four update routines implementing the Bellman-Ford relaxation
3. linkhandler0() and linkhandler1() -> included as empty stubs as required for the undergraduate version
4. Selective broadcast -> updates are only sent to neighbors
5. INF overflow guard -> prevents integer overflow when adding a link cost to an unreachable (INF) destination
6. All output requirements met -> timestamps, sender identity, packet contents, table state, and broadcast messages printed at every event
7. No global variables shared between files -> static keyword used throughout to enforce file-level scope
8. Program compiles without errors and terminates cleanly when no packets remain in the network

b. Resources Used

i. Course textbook - Computer Networking: A Top-Down Approach by Kurose & Ross, Chapter 5 (Network Layer: Control Plane), specifically the sections on Distance Vector Routing and the Bellman-Ford equation
ii. Course lecture slides on Distance Vector routing, distributed algorithms, and the relationship between Bellman-Ford and DV convergence
iii. Assignment specification and simulator provided by the instructor (Prog3.c, prototype node files)
iv. GCC compiler documentation for understanding K&R-style C function declarations used in the prototype files

c. Group Meeting

We did not have in-person meetings other than during and after class times. But during those times we:

1. Reviewed assignment spec, studied DV algorithm, divided node file responsibilities
2. Coded rtinit and rtupdate functions, resolved Prog3.c compatibility issues (float clocktime, linkhandler stubs)
3. Debugging session - identified and fixed bugs across all four node files, verified convergence output
4. Final testing with TRACE=2, wrote report, prepared submission

d. <u>Features Implemented</u>

Beyond the basic requirements, the following design choices make the implementation more robust and efficient:

Data-Driven Neighbor Logic:

Each Node file declares a static link-cost array lc[] that serves as the single source of truth for all neighbor decisions. Whether a packet should be accepted, which nodes to broadcast to, and what link cost to use in the Bellman-Ford formula all derive from this one array. No switch statements or hardcoded node IDs are needed in the algorithm logic.

Selective Broadcast:

After every table update, the new per-destination minimums are compared against a snapshot taken before the update. Broadcasts are only sent when a route minimum actually decreases, not just whenever any table entry changes. This significantly reduces unnecessary packet traffic during convergence and mirrors how real routing protocols behave.

INF Overflow Guard:

Before computing lc[from] + rcvd.mincost[d], the code checks if rcvd.mincost[d] >= INF. Without this guard, adding a link cost (example: 3) to INF (9999) would produce a value like 10002 which could incorrectly win the minimum comparison against a legitimate route cost, corrupting the distance table silently.

Uniform Code Structure:

All four node files share an identical algorithm structure. The only per-node differences are the lc[] link-cost array values and which neighbor IDs appear in the broadcast sends. This uniformity makes the code straightforward to verify for correctness and matches the real-world principle that each router runs the same software with only its local configuration differing.

7. <u>INDIVIDUAL REPORTS</u>

a. Student 1: EJ Knights

- Implemented Node0.c and Node3.c
- Contributed to the overall algorithm design for all four nodes

- wrote getmincosts() helper and selective broadcast logic
- Worked through compatibility issues between the prototype node files and the Prog3.c sim
- Spent 8-10 hours total
    - roughly 2 studying algorithms and logic
    - roughly 4 into coding and debugging
    - remaining time spent verifying and writing the report

Difficulties:

The hardest problem for me was the extern float clocktime declaration mismatch. All our initial code files we had extern int clocktime. This was before I realized there were prototype node files that I could have used. The initial int compiled without errors but produced completely wrong timestamp output at runtime. I was able to figure out that the timestamp issue came from that int to float difference. It made me realize the importance of a silent runtime corruption bug.

Learned:

This assignment gave me a much deeper understanding of how the Bellman-Ford algorithm works in a distributed setting. Reading about it in the textbook made it seem abstract, but seeing the distance tables update step-by-step in the simulation output, watching Node 0 discover the cheaper path to Node 2 through Node 1, made the algorithm click in a very concrete way. I also learned a lot about the difference between compile-time and runtime bugs in C, and how type mismatches across compilation units can produce silent incorrect behavior that is much harder to debug than a compiler error.

b. Student 2: Lei Tapungot

i. Verifying the nodes files with prog
ii. Inputted definitions on the report
iii. Spent time looking at outside resources that contributed to the program
iv. Validated prog was outputting right
v. Spent around 4-8 hours

Difficulties/Learned:

One of my difficulties was making sure Prog was working with the node codes. I noticed a problem with the output and spent time fixing that. It took roughly an hour to figure out what was wrong with the code so I took the parts out that conflicted the output I had questions that were answered by looking at more sources on the internet. There were sources that were not so helpful which took some time but I eventually found sources that positively influenced the program.

I learned more about the algorithm through the code and seeing the output go through different nodes depending on the cheaper path. Also I learned more about windows powershell and the different intricacies that it comes with. I could launch and run codes straight from the file folder, right into the powershell in order to be in the right environment.

8. SAMPLE OUTPUT (TRACE 2)

```
Enter TRACE:2

rtinit0() called at t=0.000
                via
   D0 |    1     2     3
  ----|----------------
     1|    1   9999   9999
dest 2| 9999     3   9999
     3| 9999   9999     7
rtinit0: sending to node 1: [0 1 3 7]
rinit0: sending to node 2: [0 1 3 7]
rinit0: sending to node 3: [0 1 3 7]

rtinit1() called at t=0.000
              via
   D1 |    0     2
  ----|-----------
     0|    1   9999
dest 2| 9999     1
     3| 9999   9999
rtinit1: sending to node 0: [1 0 1 9999]
rinit1: sending to node 2: [1 0 1 9999]

rtinit2() called at t=0.000
                via
   D2 |    0     1     3
  ----|----------------
     0|    3   9999   9999
dest 1| 9999     1   9999
     3| 9999   9999     2
rtinit2: sending to node 0: [3 1 0 2]
rinit2: sending to node 1: [3 1 0 2]
rinit2: sending to node 3: [3 1 0 2]
```

```
rtinit3() called at t=0.000
            via
   D3 |    0     2
  ----|-----------
     0|    7   9999
dest 1| 9999   9999
     2| 9999     2
rtinit3: sending to node 0: [7 9999 2 0]
rinit3: sending to node 2: [7 9999 2 0]
MAIN: rcv event, t=0.947, at 3 src: 0, dest: 3, contents:   0   1   3   7

rtupdate3() called at t=0.947, pkt from node 0 [0 1 3 7]
rtupdate3: table updated
            via
   D3 |    0     2
  ----|-----------
     0|    7   9999
dest 1|    8   9999
     2|   10     2
rtupdate3: min-costs changed, broadcasting [7 8 2 0]
MAIN: rcv event, t=0.992, at 0 src: 1, dest: 0, contents:   1   0   1 9999

rtupdate0() called at t=0.992, pkt from node 1 [1 0 1 9999]
rtupdate0: table updated
                via
   D0 |    1     2     3
  ----|----------------
     1|    1   9999   9999
dest 2|    2     3   9999
     3| 9999   9999     7
rtupdate0: min-costs changed, broadcasting [0 1 2 7]
MAIN: rcv event, t=1.209, at 3 src: 2, dest: 3, contents:   3   1   0   2
```

```
rtupdate3() called at t=1.209, pkt from node 2 [3 1 0 2]
rtupdate3: table updated
            via
   D3 |    0     2
  ----|-----------
     0|    7     5
dest 1|    8     3
     2|   10     2
rtupdate3: min-costs changed, broadcasting [5 3 2 0]
MAIN: rcv event, t=1.276, at 3 src: 0, dest: 3, contents:   0   1   2   7

rtupdate3() called at t=1.276, pkt from node 0 [0 1 2 7]
rtupdate3: table updated
            via
   D3 |    0     2
  ----|-----------
     0|    7     5
dest 1|    8     3
     2|    9     2
rupdate3: table changed but min-costs same, no broadcast
MAIN: rcv event, t=1.642, at 2 src: 0, dest: 2, contents:   0   1   3   7

rtupdate2() called at t=1.642, pkt from node 0 [0 1 3 7]
rtupdate2: table updated
                via
   D2 |    0     1     3
  ----|----------------
     0|    3   9999   9999
dest 1|    4     1   9999
     3|   10   9999     2
rupdate2: table changed but min-costs same, no broadcast
MAIN: rcv event, t=1.871, at 1 src: 0, dest: 1, contents:   0   1   3   7

rtupdate1() called at t=1.871, pkt from node 0 [0 1 3 7]
```

```
rtupdate0: table updated
              via
   D1 |    0     2
  ----|-----------
     0|    1   9999
dest 2|    4     1
     3|    8   9999
rtupdate1: min-costs changed, broadcasting [1 0 1 8]
MAIN: rcv event, t=2.166, at 2 src: 1, dest: 2, contents:   1   0   1 9999

rtupdate2() called at t=2.166, pkt from node 1 [1 0 1 9999]
rtupdate2: table updated
                via
   D2 |    0     1     3
  ----|----------------
     0|    3     2   9999
dest 1|    4     1   9999
     3|   10   9999     2
rtupdate2: min-costs changed, broadcasting [2 1 0 2]
MAIN: rcv event, t=2.407, at 0 src: 2, dest: 0, contents:   3   1   0   2

rtupdate0() called at t=2.407, pkt from node 2 [3 1 0 2]
rtupdate0: table updated
                via
   D0 |    1     2     3
  ----|----------------
     1|    1     4   9999
dest 2|    2     3   9999
     3| 9999     5     7
rtupdate0: min-costs changed, broadcasting [0 1 2 5]
MAIN: rcv event, t=2.421, at 2 src: 3, dest: 2, contents:   7 9999   2   0

rtupdate2() called at t=2.421, pkt from node 3 [7 9999 2 0]
rtupdate2: table updated
```

```
                via
   D2 |    0     1    3
  ----|----------------
     0|    3     2    9
dest 1|    4     1    9999
     3|   10   9999    2
rupdate2: table changed but min-costs same, no broadcast
MAIN: rcv event, t=2.811, at 1 src: 2, dest: 1, contents:   3   1   0   2

rtupdate1() called at t=2.811, pkt from node 2 [3 1 0 2]
rtupdate0: table updated
              via
   D1 |    0     2
  ----|-----------
     0|    1     4
dest 2|    4     1
     3|    8     3
rtupdate1: min-costs changed, broadcasting [1 0 1 3]
MAIN: rcv event, t=3.293, at 2 src: 3, dest: 2, contents:   7   8   2   0

rtupdate2() called at t=3.293, pkt from node 3 [7 8 2 0]
rtupdate2: table updated
                via
   D2 |    0     1    3
  ----|----------------
     0|    3     2    9
dest 1|    4     1    10
     3|   10   9999    2
rupdate2: table changed but min-costs same, no broadcast
MAIN: rcv event, t=3.602, at 3 src: 2, dest: 3, contents:   2   1   0   2

rtupdate3() called at t=3.602, pkt from node 2 [2 1 0 2]
rtupdate3: table updated
              via
```

```
              via
   D3 |    0     2
  ----|-----------
     0|    7     4
dest 1|    8     3
     2|    9     2
rtupdate3: min-costs changed, broadcasting [4 3 2 0]
MAIN: rcv event, t=4.063, at 2 src: 0, dest: 2, contents:   0   1   2   7

rtupdate2() called at t=4.063, pkt from node 0 [0 1 2 7]
rtupdate2: table updated
                via
   D2 |    0     1    3
  ----|----------------
     0|    3     2    9
dest 1|    4     1    10
     3|   10   9999    2
rupdate2: table changed but min-costs same, no broadcast
MAIN: rcv event, t=4.104, at 0 src: 3, dest: 0, contents:   7 9999   2   0

rtupdate0() called at t=4.104, pkt from node 3 [7 9999 2 0]
rtupdate0: table updated
                via
   D0 |    1     2    3
  ----|----------------
     1|    1     4   9999
dest 2|    2     3    9
     3|  9999    5    7
rupdate0: table changed but min-costs same, no broadcast
MAIN: rcv event, t=4.169, at 2 src: 3, dest: 2, contents:   5   3   2   0

rtupdate2() called at t=4.169, pkt from node 3 [5 3 2 0]
rtupdate2: table updated
```

```
                via
   D2 |    0     1    3
  ----|----------------
     0|    3     2    7
dest 1|    4     1    5
     3|   10   9999    2
rupdate2: table changed but min-costs same, no broadcast
MAIN: rcv event, t=4.330, at 0 src: 3, dest: 0, contents:   7   8   2   0

rtupdate0() called at t=4.330, pkt from node 3 [7 8 2 0]
rtupdate0: table updated
                via
   D0 |    1     2    3
  ----|----------------
     1|    1     4    15
dest 2|    2     3    9
     3|  9999    5    7
rupdate0: table changed but min-costs same, no broadcast
MAIN: rcv event, t=4.643, at 1 src: 0, dest: 1, contents:   0   1   2   7

rtupdate1() called at t=4.643, pkt from node 0 [0 1 2 7]
rtupdate0: table updated
              via
   D1 |    0     2
  ----|-----------
     0|    1     4
dest 2|    3     1
     3|    8     3
rupdate1: table changed but min-costs same, no broadcast
MAIN: rcv event, t=5.213, at 0 src: 3, dest: 0, contents:   5   3   2   0

rtupdate0() called at t=5.213, pkt from node 3 [5 3 2 0]
rtupdate0: table updated
                via
```

```
                via
   D0 |    1     2    3
  ----|----------------
     1|    1     4    10
dest 2|    2     3    9
     3|  9999    5    7
rupdate0: table changed but min-costs same, no broadcast
MAIN: rcv event, t=5.384, at 3 src: 0, dest: 3, contents:   0   1   2   5

rtupdate3() called at t=5.384, pkt from node 0 [0 1 2 5]
rtupdate3: table updated
              via
   D3 |    0     2
  ----|-----------
     0|    7     4
dest 1|    8     3
     2|    9     2
rupdate3: table changed but min-costs same, no broadcast
MAIN: rcv event, t=5.820, at 1 src: 2, dest: 1, contents:   2   1   0   2

rtupdate1() called at t=5.820, pkt from node 2 [2 1 0 2]
rtupdate0: table updated
              via
   D1 |    0     2
  ----|-----------
     0|    1     3
dest 2|    3     1
     3|    8     3
rupdate1: table changed but min-costs same, no broadcast
MAIN: rcv event, t=6.042, at 2 src: 1, dest: 2, contents:   1   0   1   8

rtupdate2() called at t=6.042, pkt from node 1 [1 0 1 8]
rtupdate2: table updated
                via
```

```
   D2 |    0     1    3
  ----|----------------
     0|    3     2    7
dest 1|    4     1    5
     3|   10     9    2
rupdate2: table changed but min-costs same, no broadcast
MAIN: rcv event, t=6.071, at 0 src: 1, dest: 0, contents:   1   0   1   8

rtupdate0() called at t=6.071, pkt from node 1 [1 0 1 8]
rtupdate0: table updated
                via
   D0 |    1     2    3
  ----|----------------
     1|    1     4   10
dest 2|    2     3    9
     3|    9     5    7
rupdate0: table changed but min-costs same, no broadcast
MAIN: rcv event, t=6.532, at 1 src: 0, dest: 1, contents:   0   1   2   5

rtupdate1() called at t=6.532, pkt from node 0 [0 1 2 5]
rtupdate0: table updated
            via
   D1 |    0     2
  ----|----------
     0|    1     3
dest 2|    3     1
     3|    6     3
rupdate1: table changed but min-costs same, no broadcast
MAIN: rcv event, t=7.021, at 0 src: 2, dest: 0, contents:   2   1   0   2

rtupdate0() called at t=7.021, pkt from node 2 [2 1 0 2]
rtupdate0: table updated
```

```
   D0 |    1     2    3
  ----|----------------
     1|    1     4   10
dest 2|    2     3    9
     3|    9     5    7
rupdate0: table changed but min-costs same, no broadcast
MAIN: rcv event, t=7.160, at 2 src: 0, dest: 2, contents:   0   1   2   5

rtupdate2() called at t=7.160, pkt from node 0 [0 1 2 5]
rtupdate2: table updated
                via
   D2 |    0     1    3
  ----|----------------
     0|    3     2    7
dest 1|    4     1    5
     3|    8     9    2
rupdate2: table changed but min-costs same, no broadcast
MAIN: rcv event, t=7.405, at 0 src: 1, dest: 0, contents:   1   0   1   3

rtupdate0() called at t=7.405, pkt from node 1 [1 0 1 3]
rtupdate0: table updated
                via
   D0 |    1     2    3
  ----|----------------
     1|    1     4   10
dest 2|    2     3    9
     3|    4     5    7
rtupdate0: min-costs changed, broadcasting [0 1 2 4]
MAIN: rcv event, t=7.579, at 3 src: 0, dest: 3, contents:   0   1   2   4

rtupdate3() called at t=7.579, pkt from node 0 [0 1 2 4]
rtupdate3: table updated
```

```
            via
   D3 |    0     2
  ----|----------
     0|    7     4
dest 1|    8     3
     2|    9     2
rupdate3: table changed but min-costs same, no broadcast
MAIN: rcv event, t=7.941, at 1 src: 0, dest: 1, contents:   0   1   2   4

rtupdate1() called at t=7.941, pkt from node 0 [0 1 2 4]
rtupdate0: table updated
            via
   D1 |    0     2
  ----|----------
     0|    1     3
dest 2|    3     1
     3|    5     3
rupdate1: table changed but min-costs same, no broadcast
MAIN: rcv event, t=8.086, at 0 src: 3, dest: 0, contents:   4   3   2   0

rtupdate0() called at t=8.086, pkt from node 3 [4 3 2 0]
rtupdate0: table updated
                via
   D0 |    1     2    3
  ----|----------------
     1|    1     4   10
dest 2|    2     3    9
     3|    4     5    7
rupdate0: table changed but min-costs same, no broadcast
MAIN: rcv event, t=8.639, at 2 src: 1, dest: 2, contents:   1   0   1   3

rtupdate2() called at t=8.639, pkt from node 1 [1 0 1 3]
rtupdate2: table updated
```

```
rtupdate2: table updated
                via
   D2 |    0     1    3
  ----|----------------
     0|    3     2    7
dest 1|    4     1    5
     3|    8     4    2
rupdate2: table changed but min-costs same, no broadcast
MAIN: rcv event, t=8.943, at 2 src: 3, dest: 2, contents:   4   3   2   0

rtupdate2() called at t=8.943, pkt from node 3 [4 3 2 0]
rtupdate2: table updated
                via
   D2 |    0     1    3
  ----|----------------
     0|    3     2    6
dest 1|    4     1    5
     3|    8     4    2
rupdate2: table changed but min-costs same, no broadcast
MAIN: rcv event, t=9.960, at 2 src: 0, dest: 2, contents:   0   1   2   4

rtupdate2() called at t=9.960, pkt from node 0 [0 1 2 4]
rtupdate2: table updated
                via
   D2 |    0     1    3
  ----|----------------
     0|    3     2    6
dest 1|    4     1    5
     3|    7     4    2
rupdate2: table changed but min-costs same, no broadcast

Simulator terminated at t=9.959651, no packets in medium
```

9. SAMPLE OUTPUT 2

```
Enter TRACE:1

rtinit0() called at t=0.000
                via
   D0 |    1     2    3
  ----|-----------------
     1|    1   9999   9999
dest 2|  9999     3   9999
     3|  9999   9999     7
rtinit0: sending to node 1: [0 1 3 7]
rinit0: sending to node 2: [0 1 3 7]
rinit0: sending to node 3: [0 1 3 7]

rtinit1() called at t=0.000
             via
   D1 |    0     2
  ----|-----------
     0|    1   9999
dest 2|  9999     1
     3|  9999   9999
rtinit1: sending to node 0: [1 0 1 9999]
rinit1: sending to node 2: [1 0 1 9999]

rtinit2() called at t=0.000
                via
   D2 |    0     1    3
  ----|-----------------
     0|    3   9999   9999
dest 1|  9999     1   9999
     3|  9999   9999     2
rtinit2: sending to node 0: [3 1 0 2]
rinit2: sending to node 1: [3 1 0 2]
rinit2: sending to node 3: [3 1 0 2]

rtinit3() called at t=0.000
             via
   D3 |    0     2
  ----|-----------
     0|    7   9999
dest 1|  9999   9999
     2|  9999     2
```

```
rtinit3: sending to node 0: [7 9999 2 0]
rinit3: sending to node 2: [7 9999 2 0]

rtupdate1() called at t=0.000, pkt from node 0 [0 1 3 7]
rtupdate0: table updated
             via
   D1 |    0     2
  ----|-----------
     0|    1   9999
dest 2|    4     1
     3|    8   9999
rtupdate1: min-costs changed, broadcasting [1 0 1 8]

rtupdate1() called at t=0.000, pkt from node 2 [3 1 0 2]
rtupdate0: table updated
             via
   D1 |    0     2
  ----|-----------
     0|    1     4
dest 2|    4     1
     3|    8     3
rtupdate1: min-costs changed, broadcasting [1 0 1 3]

rtupdate0() called at t=0.000, pkt from node 1 [1 0 1 9999]
rtupdate0: table updated
                via
   D0 |    1     2    3
  ----|-----------------
     1|    1   9999   9999
dest 2|    2     3   9999
     3|  9999   9999     7
rtupdate0: min-costs changed, broadcasting [0 1 2 7]

rtupdate3() called at t=0.000, pkt from node 0 [0 1 3 7]
rtupdate3: table updated
             via
   D3 |    0     2
  ----|-----------
     0|    7   9999
dest 1|    8   9999
     2|   10     2
```

```
rtupdate3: min-costs changed, broadcasting [7 8 2 0]

rtupdate2() called at t=0.000, pkt from node 0 [0 1 3 7]
rtupdate2: table updated
                via
   D2 |    0     1    3
  ----|-----------------
     0|    3   9999   9999
dest 1|    4     1   9999
     3|   10   9999     2
rupdate2: table changed but min-costs same, no broadcast

rtupdate0() called at t=0.000, pkt from node 2 [3 1 0 2]
rtupdate0: table updated
                via
   D0 |    1     2    3
  ----|-----------------
     1|    1     4   9999
dest 2|    2     3   9999
     3|  9999     5     7
rtupdate0: min-costs changed, broadcasting [0 1 2 5]

rtupdate3() called at t=0.000, pkt from node 2 [3 1 0 2]
rtupdate3: table updated
              via
   D3 |    0     2
  ----|-----------
     0|    7     5
dest 1|    8     3
     2|   10     2
rtupdate3: min-costs changed, broadcasting [5 3 2 0]

rtupdate0() called at t=0.000, pkt from node 3 [7 9999 2 0]
rtupdate0: table updated
                via
   D0 |    1     2    3
  ----|-----------------
     1|    1     4   9999
dest 2|    2     3     9
     3|  9999     5     7
rupdate0: table changed but min-costs same, no broadcast
```

```
rtupdate0() called at t=0.000, pkt from node 1 [1 0 1 8]
rtupdate0: table updated
                via
   D0 |    1     2    3
  ----|-----------------
     1|    1     4   9999
dest 2|    2     3     9
     3|    9     5     7
rupdate0: table changed but min-costs same, no broadcast

rtupdate2() called at t=0.000, pkt from node 1 [1 0 1 9999]
rtupdate2: table updated
                via
   D2 |    0     1    3
  ----|-----------------
     0|    3     2   9999
dest 1|    4     1   9999
     3|   10   9999     2
rtupdate2: min-costs changed, broadcasting [2 1 0 2]

rtupdate1() called at t=0.000, pkt from node 0 [0 1 2 7]
rtupdate0: table updated
              via
   D1 |    0     2
  ----|-----------
     0|    1     4
dest 2|    3     1
     3|    8     3
rupdate1: table changed but min-costs same, no broadcast

rtupdate0() called at t=0.000, pkt from node 1 [1 0 1 3]
rtupdate0: table updated
                via
   D0 |    1     2    3
  ----|-----------------
     1|    1     4   9999
dest 2|    2     3     9
     3|    4     5     7
rtupdate0: min-costs changed, broadcasting [0 1 2 4]
```

```
rtupdate3() called at t=0.000, pkt from node 0 [0 1 2 7]
rtupdate3: table updated
             via
   D3 |    0     2
  ----|-----------
     0|    7     5
dest 1|    8     3
     2|    9     2
rupdate3: table changed but min-costs same, no broadcast

rtupdate2() called at t=0.000, pkt from node 3 [7 9999 2 0]
rtupdate2: table updated
                via
   D2 |    0     1    3
  ----|-----------------
     0|    3     2     9
dest 1|    4     1   9999
     3|   10   9999     2
rupdate2: table changed but min-costs same, no broadcast

rtupdate0() called at t=0.000, pkt from node 3 [7 8 2 0]
rtupdate0: table updated
                via
   D0 |    1     2    3
  ----|-----------------
     1|    1     4    15
dest 2|    2     3     9
     3|    4     5     7
rupdate0: table changed but min-costs same, no broadcast

rtupdate1() called at t=0.000, pkt from node 0 [0 1 2 5]
rtupdate0: table updated
             via
   D1 |    0     2
  ----|-----------
     0|    1     4
dest 2|    3     1
     3|    6     3
rupdate1: table changed but min-costs same, no broadcast
```

```
rtupdate1() called at t=0.000, pkt from node 2 [2 1 0 2]
rtupdate0: table updated
               via
   D1 |     0      2
  ----|------------
     0|     1      3
dest 2|     3      1
     3|     6      3
rupdate1: table changed but min-costs same, no broadcast

rtupdate2() called at t=0.000, pkt from node 1 [1 0 1 8]
rtupdate2: table updated
                 via
   D2 |     0      1     3
  ----|-----------------
     0|     3      2      9
dest 1|     4      1    9999
     3|    10      9      2
rupdate2: table changed but min-costs same, no broadcast

rtupdate3() called at t=0.000, pkt from node 0 [0 1 2 5]
rtupdate3: table updated
               via
   D3 |     0      2
  ----|-----------
     0|     7      5
dest 1|     8      3
     2|     9      2
rupdate3: table changed but min-costs same, no broadcast

rtupdate0() called at t=0.000, pkt from node 3 [5 3 2 0]
rtupdate0: table updated
                 via
   D0 |     1      2     3
  ----|-----------------
     1|     1      4    10
dest 2|     2      3     9
     3|     4      5     7
rupdate0: table changed but min-costs same, no broadcast

rtupdate2() called at t=0.000, pkt from node 1 [1 0 1 3]
```

```
rtupdate2: table updated
                via
   D2 |    0     1    3
  ----|-----------------
     0|    3     2     9
dest 1|    4     1   9999
     3|   10     4     2
rupdate2: table changed but min-costs same, no broadcast

rtupdate1() called at t=0.000, pkt from node 0 [0 1 2 4]
rtupdate0: table updated
              via
   D1 |    0     2
  ----|-----------
     0|    1     3
dest 2|    3     1
     3|    5     3
rupdate1: table changed but min-costs same, no broadcast

rtupdate3() called at t=0.000, pkt from node 2 [2 1 0 2]
rtupdate3: table updated
              via
   D3 |    0     2
  ----|-----------
     0|    7     4
dest 1|    8     3
     2|    9     2
rtupdate3: min-costs changed, broadcasting [4 3 2 0]

rtupdate0() called at t=0.000, pkt from node 2 [2 1 0 2]
rtupdate0: table updated
                via
   D0 |    1     2    3
  ----|-----------------
     1|    1     4    10
dest 2|    2     3     9
     3|    4     5     7
rupdate0: table changed but min-costs same, no broadcast

rtupdate2() called at t=0.000, pkt from node 0 [0 1 2 7]
rtupdate2: table updated
```

```
                via
   D2 |    0     1    3
  ----|-----------------
     0|    3     2     9
dest 1|    4     1   9999
     3|   10     4     2
rupdate2: table changed but min-costs same, no broadcast

rtupdate3() called at t=0.000, pkt from node 0 [0 1 2 4]
rtupdate3: table updated
              via
   D3 |    0     2
  ----|-----------
     0|    7     4
dest 1|    8     3
     2|    9     2
rupdate3: table changed but min-costs same, no broadcast

rtupdate0() called at t=0.000, pkt from node 3 [4 3 2 0]
rtupdate0: table updated
                via
   D0 |    1     2    3
  ----|-----------------
     1|    1     4    10
dest 2|    2     3     9
     3|    4     5     7
rupdate0: table changed but min-costs same, no broadcast

rtupdate2() called at t=0.000, pkt from node 3 [7 8 2 0]
rtupdate2: table updated
                via
   D2 |    0     1    3
  ----|-----------------
     0|    3     2     9
dest 1|    4     1    10
     3|   10     4     2
rupdate2: table changed but min-costs same, no broadcast

rtupdate2() called at t=0.000, pkt from node 0 [0 1 2 5]
rtupdate2: table updated
                via
```

```
   D2 |    0     1    3
  ----|----------------
     0|    3     2     9
dest 1|    4     1    10
     3|    8     4     2
rupdate2: table changed but min-costs same, no broadcast

rtupdate2() called at t=0.000, pkt from node 3 [5 3 2 0]
rtupdate2: table updated
                via
   D2 |    0     1    3
  ----|----------------
     0|    3     2     7
dest 1|    4     1     5
     3|    8     4     2
rupdate2: table changed but min-costs same, no broadcast

rtupdate2() called at t=0.000, pkt from node 0 [0 1 2 4]
rtupdate2: table updated
                via
   D2 |    0     1    3
  ----|----------------
     0|    3     2     7
dest 1|    4     1     5
     3|    7     4     2
rupdate2: table changed but min-costs same, no broadcast

rtupdate2() called at t=0.000, pkt from node 3 [4 3 2 0]
rtupdate2: table updated
                via
   D2 |    0     1    3
  ----|----------------
     0|    3     2     6
dest 1|    4     1     5
     3|    7     4     2
rupdate2: table changed but min-costs same, no broadcast

Simulator terminated at t=0.000168, no packets in medium
```